L11: Metalinguistic Abstraction I

CS1101S: Programming Methodology

Martin Henz

November 1, 2023

Module Overview

- Unit 1—Functions (textbook Chapter 1)
  - Getting acquainted with programming, using functional abstraction
  - Learning to read programs, statically, and using the substitution model
  - Example applications: runes, curves
- Unit 2—Data (textbook Chapter 2)
  - Getting familiar with data: pairs, lists, trees
  - Searching in lists and trees, sorting of lists
  - Example application: sound processing

Module Overview, continued

- Unit 3—State (parts of textbook Chapter 3)
  - Programming with stateful abstractions
  - Arrays, loops, searching in and sorting of arrays
  - Reading programs using the **CSE machine**
  - Example applications: robots, video processing
- Unit 4—Beyond (parts of textbook Chapters 3 and 4)
  - Streams (example application: video stream processing)
  - **Metalinguistic abstraction: understanding the CSE machine by *programming it***

The last three semester weeks

Week 11:
- Lecture L11: Metalinguistic Abstraction I
- Brief B11: Metalinguistic Abstraction II
- Practical Assessment: Saturday 4/11, **look out for our announcements**

Week 12:
- Lecture L12A: Logic Programming
- Lecture L12B: Concurrent Programming
- Lecture L12C: Register Machines
- Lecture L12D: Meta-circular Evaluator
- No Brief B12: NUS Well Being Day

Week 13:
- Lecture L13: The Power of Simplicity (selected solutions to Missions and Quests)
- Brief B13: Final Brief

Final Assessment Scope: All material until (including) Brief B11, *except* Control (C) and Stash (S)

CP3108: A project module on "Source Academy"

Time and workload: Sem 2 AY2023/24; CP3108A/B (2/4 Units)

Application: from today to Friday Week 13 via this form

Interviews: 20/11–8/12/2023

Scope: topics listed in Github; self-proposed projects are possible

Avenger recruitment

Applications: from 17/11/2023 (Friday Week 13) to 9/1/2023 (Monday Week 1), look out for announcements

Interviews: throughout Sem 2

Selection: first round before end of Sem 2; waiting list

Why write a CSE machine in Source?

Recall: programming is...

...communicating computational processes

By now...

...we are quite effective in using Source for communicating computational processes

Consider the CSE machine

The machine describes a computational process (control, stash, frames etc) that unfolds *whenever any* program runs.

Our goal today

Can we use Source to better understand the CSE machine?

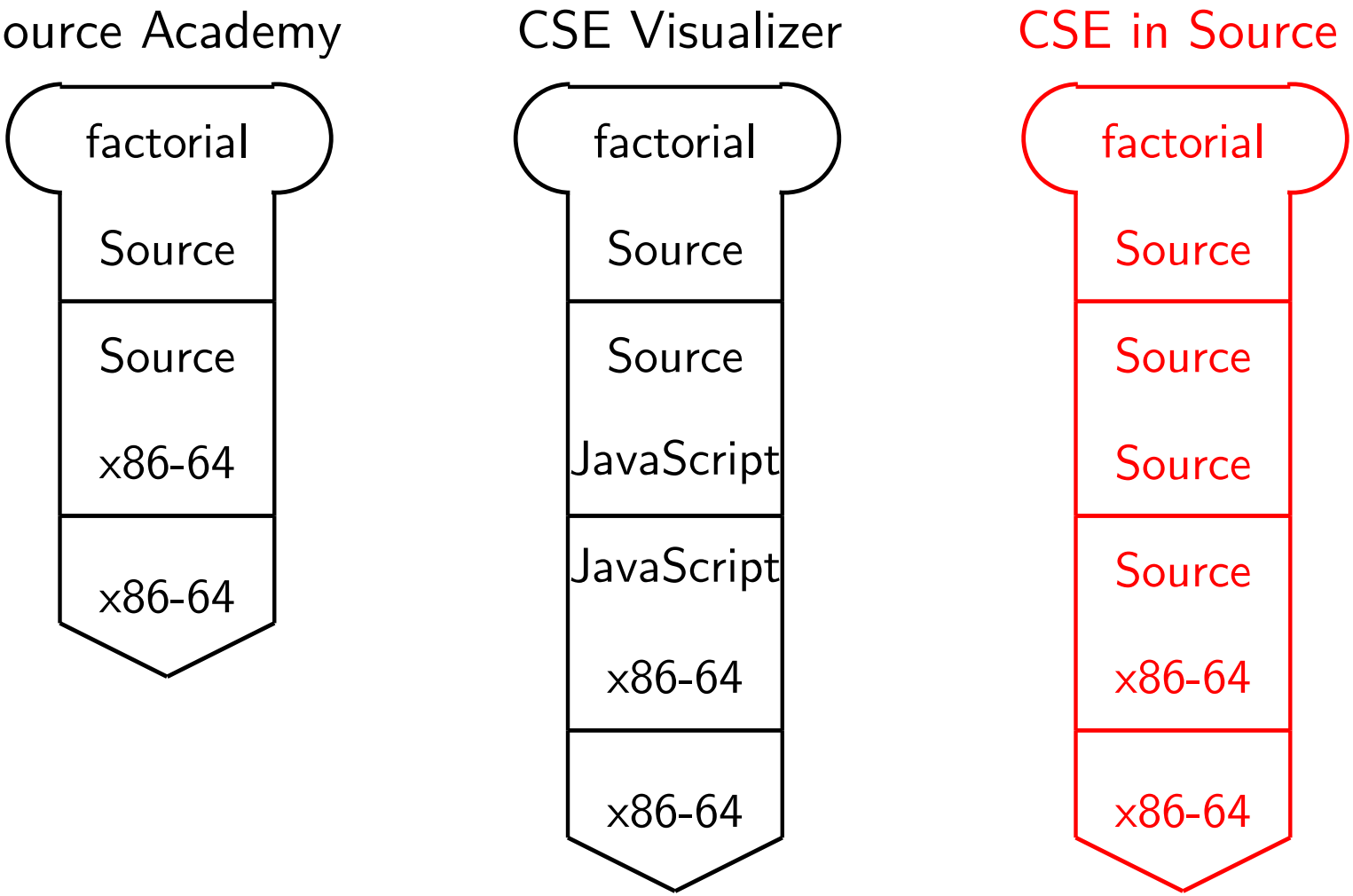
Importance of metalinguistic abstraction

The most fundamental idea in programming

The evaluator, which determines the meaning of statements and expressions in a programming language, is just another program.

- Symbolic evaluator/differentiator in Lecture L6 is just another program!
- Source Academy where your programs run is just another program!
- Firmware that you uploaded to the EV3 LEGO Mindstorms computers is just another program!
- CSE machine visualizer is just another program!

Ways to run `factorial` in Source Academy



Our task and the objective

Task

We want to write an evaluator in Source that can execute programs that are written in Source

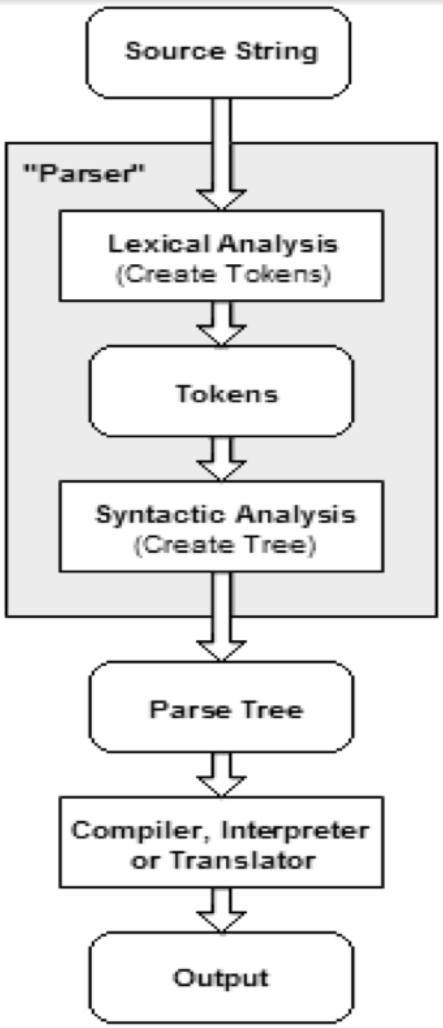
Why?

There is no *practical* use for such a CSE machine.
After all, in order to run the evaluator, you need to have already a working implementation of Source!

Objective

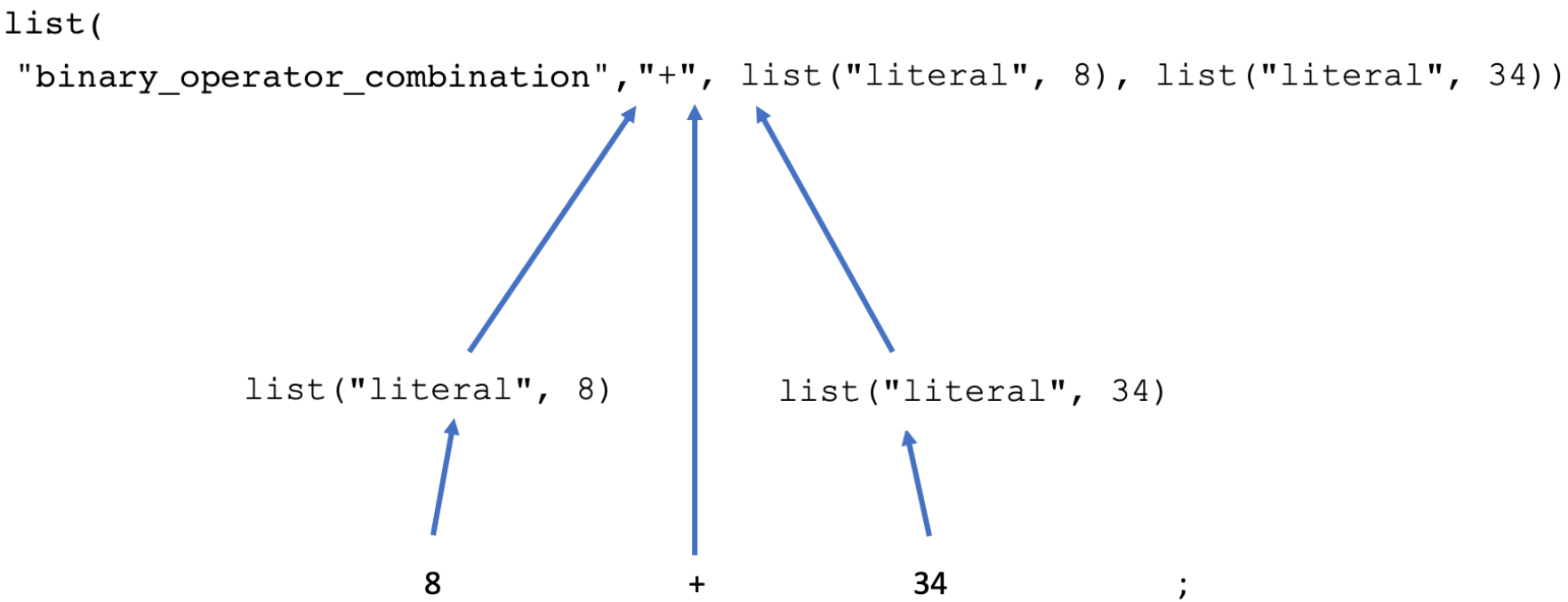
By writing a CSE machine in Source, we hope to gain new insights into the language and a deeper understanding of the machine.

First step for evaluator: parsing



- Lexical analysis
  - Input characters are split into meaningful symbols (tokens) defined using *regular expressions*
  - Example: `"8 + 34;"` is split into tokens `8` `+` `34` `;`
- Parsing
  - Checks that the tokens form allowable *components* (statements or expressions)
  - Produces symbolic representation of the program, a *syntax tree*

Syntax tree in graphical notation



Support for parsing in Source §4

```
const syntax_tree
    = parse("8 + 34;");
```

generates the syntax tree as Source list:

```
display_list(syntax_tree);
```

displays

```
list("binary_operator_combination",
     "+",
     list("literal", 8),
     list("literal", 34))
```

Scope of L11

L11: evaluator for Source §1 (simple functions)

Sequence of evaluators describing the structure and interpretation of Source §1, starting with a "Calculator language"

Outlook

Brief B11 covers complex functions and loops

Reading

- SICP JS 4.1.2, 4.1.3, 4.1.4 (skim 4.1.1)

Calculator language in Backus-Naur Form (BNF)

stmt	::=	*expr* ;	expression statement
expr	::=	*expr bin-op expr*	binary operator combination
	|	*number*	number expression
	|	(*expr*)	parenthesised expression
bin-op	::=	+ | - | * | /	binary operator

Example:

```
1.4 + 2.3 / 70.4;
```

Parsing the calculator language

To run the program

```
1.4 + 2.3 / 70.4;
```

we first parse it:

```
parse("1.4 + 2.3 / 70.4;");
```

returns the syntax tree

```
list("binary_operator_combination",
     "+",
     list("literal", 1.4),
     list("binary_operator_combination", "/",
          list("literal", 2.3),
          list("literal", 70.4)))
```

Syntax of literals

Example: `parse("... 2.3 ...");` ⇒
`...list("literal", 2.3)...`

```
// syntax predicate, in the following in maroon
function is_literal(comp) {
  return is_tagged_list(comp, "literal");
}
function is_tagged_list(comp, the_tag) {
  return is_pair(comp) && head(comp) === the_tag;
}
// selector, in the following in blue
function literal_value(comp) {
  return head(tail(comp));
}
```

Syntax of operator combinations

```
parse("... 2.3 + 70.4 ...");⇒

... list("binary_operator_combination", "+",
         list("literal", 2.3),
         list("literal", 70.4)) ...

// syntax predicate (parsing details in 4.1.2)
const is_bin_op_combination = comp =>
is_tagged_list(comp,"binary_operator_combination");
// selectors
const operator_symbol = comp => list_ref(comp,1);
const first_operand   = comp => list_ref(comp,2);
const second_operand  = comp => list_ref(comp,3);
```

Structure of CS machine for calculator expressions

```
function evaluate(expr) {
  let C = list(expr);
  let S = null;
  while (! is_null(C)) {
    const command = head(C); C = tail(C);
    if (is_literal(command)) {
      S = pair(literal_value(command), S);
    } else if (...) { // handle all other commands
    } else { error("unknown command"); }
  }
  return head(S);
}
```

Evaluating literal expressions

Example: `2.3;`

```
Control: 2.3
Stash:

==>

Control:
Stash: 2.3

==>

exit while loop, result on stash
```

Evaluating operator combinations

Example: `2.3 / 70.4`

```
...} else if (is_bin_op_combination(command)) {
  C = pair(first_operand(command),
           pair(second_operand(command),
             pair(make_bin_op_instr(
               operator_symbol(command)),
                 C)));
} else if (is_bin_op_instr(command)) {
  S = pair(apply_binary(op_instr_symbol(command),
                        head(tail(S)), head(S)),
           tail(tail(S)));
} ...
// instr syntax in green, instr selector in purple
```

Evaluating operator combinations

Example: `2.3 / 70.4`

```
function apply_binary(operator, op1, op2) {
    return operator === "+"
           ? op1 + op2
           : operator === "-"
           ? op1 - op2
           : operator === "*"
           ? op1 * op2
           : operator === "/"
           ? op1 / op2
           : error("unknown operator");
}
```

Adding booleans, conditionals, and sequences

stmt	::=	$\cdots$	
	$\mid$	$stmt_1$ `...` $stmt_n$	statement sequence
expr	::=	$\cdots$	
	$\mid$	`true` $\mid$ `false`	boolean literals
	$\mid$	$expr_1$ `?` $expr_2$ `:` $expr_3$	conditional expression

Example:

```
8 + 34; true ? 1 + 2 : 17;
```

Evaluating the language

```
function evaluate(program) {
  let C = list(program); let S = null;
  while (! is_null(C)) {
    const command = head(C); C = tail(C);
    if ... // new expressions and statements
    } else if (is_conditional(command)) { ...
    } else if (is_sequence(command)) { ...
           // new instructions
    } else if (is_branch_instr(command)) { ...
    } else if (is_pop_instr(command)) { ...
    } else {
      error("unknown command"); } }
  return head(S); }
```

Parsing conditionals: an example

```
parse("true ? 42 : 17;");
```

returns

```
list("conditional_expression",
     list("literal", true),
     list("literal", 42),
     list("literal", 17))
```

Syntax predicate: `is_conditional`
Selectors: `cond_predicate`, `cond_consequent`, `cond_alternative`

Evaluating conditionals

Example: `true ? 42 : 17;`

```
while (! is_null(C)) {
  const command = head(C); C = tail(C);
  if ... else if (is_conditional(command)) {
    C = pair(cond_predicate(command),
             pair(make_branch_instr(
                    cond_consequent(command),
                    cond_alternative(command)),
                  C));
  } else if ...
} }
```

Evaluating branch instructions

Example: `branch 42 17`

```
while (! is_null(C)) {
  const command = head(C); C = tail(C);
  if ... else if (is_branch_instr(command)) {
    C = pair(is_truthy(head(S))
             ? branch_instr_consequent(command)
             : branch_instr_alternative(command),
             C);
    S = tail(S);
  } else if ...
} }
const is_truthy = x => is_boolean(x) ? x : error();
```

Parsing sequences: an example

```
parse("1; 2; 3;");
```

returns

```
list("sequence",
     list(list("literal", 1),
          list("literal", 2),
          list("literal", 3))
```

Syntax predicate: `is_sequence`
Selector: `sequence_statements`

Evaluating sequences

Example: `1; 2; 3;`

```
while (! is_null(C)) {
  const command = head(C); C = tail(C);
  if ... else if (is_sequence(command)) {
    const body = sequence_statements(command);
    C = is_null(body)
        ? pair(make_literal(undefined), C)
        : is_null(tail(body))
        ? pair(head(body), C)
        : pair(head(body),
            pair(make_pop_instr(),
              pair(make_sequence(tail(body)),
                C)));
  } else if ...
```

Evaluating pop instructions

Example: pop

```
while (! is_null(C)) {
  const command = head(C); ...
  if ... else if (is_pop_instr(command)) {
    S = tail(S);
  } else if ...
} }
```

Adding blocks and declarations

stmt	::=	···	
	|	{ *stmt* }	block
	|	**const** *name* = *expr* ;	constant declaration
expr	::=	···	
	|	*name*	name

Example:

```
const y = 4;
{
    const x = y + 7;
    x * 2;
}
```

Evaluating the language

```
function evaluate(program) {
  let C = list(program); let S = null;
  while (! is_null(C)) {
    const command = head(C); ...
    if ... // new expressions and statements
    } else if (is_name(command)) { ...
    } else if (is_block(command)) { ...
    } else if (is_declaration(command)) { ...
           // new instructions
    } else if (is_assign_instr(command)) { ...
    } else if (is_env_instr(command)) { ...
    } else {
      error("unknown command"); } }
...}
```

Frames and environments

```
function make_frame(names, values) {
    return pair(names, values);
}
const frame_names = head;
const frame_values = tail;

function extend_environment(ns, vs, e) {
    return pair(make_frame(ns, vs), e);
}
const the_empty_environment = null;
const the_global_environment
                    = the_empty_environment;
```

Evaluating blocks

Example: `{ const x = y + 7; x * 2; }`

```
while (! is_null(C)) {
   const command = head(C); C = tail(C);
   if ... else if (is_block(command)) {
      const locals = scan_out_declarations(
                        block_body(command));
      const unassigneds = list_of_unassigned(locals);
      C = pair(block_body(command),
            pair(make_env_instr(E), C));
      E = extend_environment(locals, unassigneds, E);
   } else if ...
   }
}
```

Scanning out declarations

Example of block statement:

```
{ const x = y; { const y = 2; } const z = x/2; x; }
```

```
function scan_out_declarations(comp) {
    return is_sequence(comp)
           ? accumulate(
                 append,
                 null,
                 map(scan_out_declarations,
                     sequence_statements(comp)))
           : is_declaration(comp)
           ? list(decl_symbol(comp))
           : null;
}
```

Evaluating declarations

Example: `const x = y + 7;`

```
while (! is_null(C)) {
  const command = head(C); C = tail(C);
  if ... else if (is_declaration(command)) {
    C = pair(make_assignment(
                  decl_symbol(command),
                  decl_value_expr(command)),
                C);
  } else if ...
} }
```

Evaluating asssignments

Example: `x = y + 7;`

```
while (! is_null(C)) {
  const command = head(C); C = tail(C);
  if ... else if (is_assignment(command)) {
    C = pair(assignment_value_expression(command),
             pair(make_assign_instr(
                    assignment_symbol(command)),
                  C));
  } else if ...
} }
```

Evaluating asssignment instructions

Example: `asgn x`

```
while (! is_null(C)) {
  const command = head(C); ...
  if ... else if (is_assign_instr(command)) {
            assign_symbol_value(
               assign_instr_symbol(command),
               head(S),
               E);
  } else if ...
} }
```

Assign a name to a value

Example: `asgn x ⇒ assign_symbol_value("x", 42, env)`

```
function assign_symbol_value(symbol, val, env) {
  function env_loop(env)                        {
    function scan(symbols, vals)                {
      return is_null(symbols)
        ? env_loop(enclosing_environment(env))
        : symbol === head(symbols)
        ? set_head(vals, val)
        : scan(tail(symbols), tail(vals)); }
    const frame = first_frame(env);
    return scan(frame_symbols(frame),
                frame_values(frame));          }
  return env_loop(env);                          }
```

Evaluating names

Example: `x`

```
while (! is_null(C)) {
  const command = head(C); ...
  if ... else if (is_name(command)) {
    S = pair(lookup_symbol_value(
                symbol_of_name(command),
                E),
             S);
  } else if ...
} }
```

Some more environment functions

```
function first_frame(env) {
    return head(env);
}

function enclosing_environment(env) {
    return tail(env);
}

function is_empty_environment(env) {
    return is_null(env);
}
```

Looking up a symbol

```
function lookup_symbol_value(symbol, env) {
  function env_loop(env) {
    function scan(symbols, vals) {
      return is_null(symbols)
             ? env_loop(enclosing_environment(env))
             : symbol === head(symbols)
             ? head(vals)
             : scan(tail(symbols),
                    tail(vals));
    }
    const frame = first_frame(env);
    return scan(frame_symbols(frame),
                frame_values(frame));
  }
  return env_loop(env);
}
```

Adding simple compound functions

stmt	::=	···	
	|	**function** *name* **(** *params* **) {**	
		return *expr* **; }**	function declaration
expr	::=	···	
	|	*expr*($expr_1$, ..., $expr_n$)	function application
	|	*params* **=>** *expr*	lambda expression

Example:

```
function fact(n) {
    return n === 1 ? 1 : n * fact(n - 1);
}
fact(5); // result: 120
```

Evaluating the language

```
function evaluate(program) {
  let C = list(program); let S = null;
  while (! is_null(C)) {
    const command = head(C); ...
    if ... // new expressions and statements
    } else if (is_lambda_expr(command)) { ...
    } else if (is_function_decl(command)) { ...
    } else if (is_application(command)) { ...
           // new instructions
    } else if (is_call_instr(command)) { ...
    } else {
      error("unknown command"); } }
  return head(S); }
```

Evaluating compound functions

```
while (! is_null(C)) {
  const command = head(C); ...
  if ... else if (is_lambda_expr(command)) {
    if (is_return_stmt(lambda_body(command))) {
      S = pair(make_simple_function(
                lambda_parameter_symbols(command),
                return_expr(lambda_body(command)),
                E), S);
    } else {
      error("body must be return statement");
    }
  } else if ...
} }
```

Function objects

Example: `x => x + y`

```
function make_simple_function(params, body, env) {
    return list("simple_function",
                parameters, body, env);
}
```

Transformation of function declaration

```
function fact(n) {
    return n === 1 ? 1 : n * fact(n - 1);
} // becomes
const fact = n => n === 1 ? 1 : n * fact(n - 1);

while (! is_null(C)) {
  const command = head(C); C = tail(C);
  if ... else if (is_function_decl(command)) {
    C = pair(function_decl_to_constant_decl(
                                        command),
             C);
  } else if ...
} }
```

Transformation of function declaration

```
function fact(n) {
    n === 1 ? 1 : n * fact(n - 1);
} // becomes
const fact = n => { n===1 ? 1 : n * fact(n - 1); };

function function_decl_to_constant_decl(stmt) {
  return make_constant_declaration(
            function_decl_name(stmt),
            make_lambda_expression(
              function_decl_parameters(stmt),
              function_decl_body(stmt)));
}
```

Function application

Example: `fact(n - 1)`

```
while (! is_null(C)) {
  const command = head(C); C = tail(C);
  if ... else if (is_application(command)) {
    const arg_exprs = arg_expressions(command);
    C = pair(function_expression(command),
             append(arg_exprs,
                    pair(make_call_instr(
                           length(arg_exprs)), C)));
  } else if ...
} }
```

Evaluation of call instructions

Example: `call 1`

```
while (! is_null(C)) {
  const command = head(C); C = tail(C);
  if ... else if (is_call_instr(command)) {
    const arity = call_instr_arity(command);
    let args = null;
    let n = arity;
    while (n > 0) {
      args = pair(head(S), args);
      S = tail(S);
      n = n - 1; }
    const fun = head(S); S = tail(S);
    ...
```

Evaluation of call instructions

Example: `call 1`

```
      ...
      const fun = head(S); S = tail(S);
      if (is_primitive_function(fun)) {
        S = pair(apply_prim_function(fun, args), S);
      } else if (is_simple_function(fun)) {
        C = pair(function_body(fun),
                 pair(make_env_instr(E), C));
        E = extend_environment(function_params(fun),
              args, function_environment(fun));
      } else { error("unknown function type"); }
    } else if ...
} }
```

Declaring primitive functions

```
const primitive_functions =
  list(pair("math_pow", math_pow,
       ...);
function setup_environment () {
    ...primitive_functions...
}
const the_global_environment =
    setup_environment();
```

Applying primitive functions

```
function apply_prim_function(fun, arglist) {
    return apply_in_underlying_javascript(
                primitive_implementation(fun),
                arglist);
}
```

Summary

- CSE machine in Source:
  just another program, communicating a computational process
- Programs, control, stash, environment: *data structures*
- Control: lists of commands (expressions, statements, instructions)
- Function objects: lists containing parameters, body, environment
- Stash: lists of values (including function objects)
- Environments: lists of pairs